

NAME- PAVNI GULYANI

BATCH -16

SAPID-590033879

PROGRAM 1: Convert Seconds into Days, Hours, Minutes and Seconds

1. Aim

To write a C program to input time in seconds and convert it into days, hours, minutes, and seconds format.

2. Algorithm

1. Start.
2. Declare integer variables totalSeconds, days, hours, minutes, and seconds.
3. Input the total time in seconds.
4. Calculate days using:
$$\text{days} = \text{totalSeconds} / 86400$$
5. Find the remaining seconds using:
$$\text{remaining} = \text{totalSeconds} \% 86400$$
6. Calculate hours using:
$$\text{hours} = \text{remaining} / 3600$$
7. Update the remaining seconds.
8. Calculate minutes using:
$$\text{minutes} = \text{remaining} / 60$$
9. Calculate the remaining seconds.
10. Display days, hours, minutes, and seconds.
11. Stop.

3. Pseudocode

BEGIN

INPUT totalSeconds

$\text{days} \leftarrow \text{totalSeconds} / 86400$

$\text{remaining} \leftarrow \text{totalSeconds} \% 86400$

$\text{hours} \leftarrow \text{remaining} / 3600$

$\text{remaining} \leftarrow \text{remaining} \% 3600$

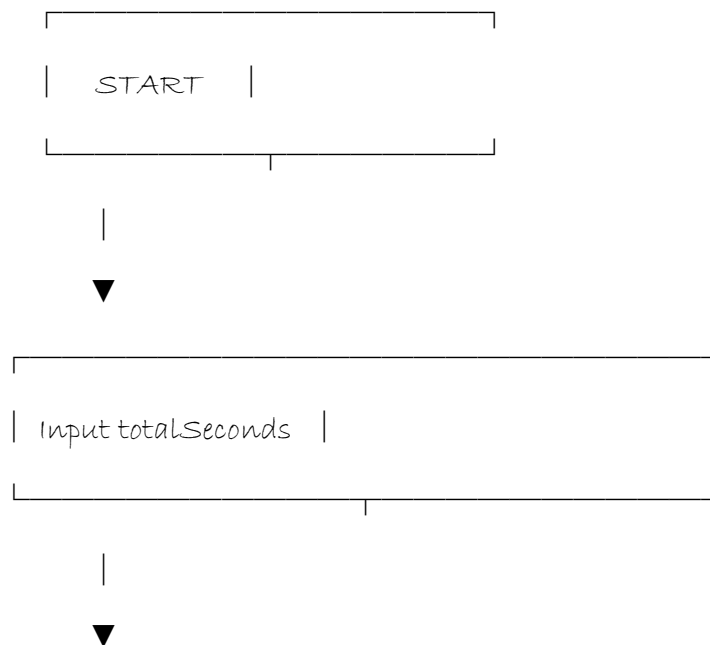
$\text{minutes} \leftarrow \text{remaining} / 60$

$\text{seconds} \leftarrow \text{remaining} \% 60$

DISPLAY days, hours, minutes, seconds

END

4. Flowchart



```
| days = totalSeconds |  
| / 86400 |  
|
```

|



```
| remaining = |  
| totalSeconds % 86400 |  
|
```

|



```
| hours = remaining |  
| / 3600 |  
|
```

|



```
| remaining = |  
| remaining % 3600 |  
|
```

|



```
|
```



5. C program

```
#include <stdio.h>
```

```
int main()
```

```

{
    int totalSeconds, days, hours, minutes, seconds, remaining;

    printf("Enter time in seconds: ");
    scanf("%d", &totalSeconds);

    days = totalSeconds / 86400;
    remaining = totalSeconds % 86400;

    hours = remaining / 3600;
    remaining = remaining % 3600;

    minutes = remaining / 60;
    seconds = remaining % 60;

    printf("\nTime = %d days, %d hours, %d minutes, %d seconds\n",
           days, hours, minutes, seconds);

    return 0;
}

```

6. Test Case Table

Test Case Input (seconds) Expected Output

TC1	90061	1 day, 1 hour, 1 minute, 1 second
TC2	86400	1 day, 0 hours, 0 minutes, 0 seconds

Test Case Input (seconds) Expected Output

TC3	3661	0 days, 1 hour, 1 minute, 1 second
TC4	60	0 days, 0 hours, 1 minute, 0 seconds
TC5	0	0 days, 0 hours, 0 minutes, 0 seconds

7. Compilation

Save the program as:

time.c

Compile using:

gcc time.c -o time

8. Execution

Run using:

./time

Sample Execution

Enter time in seconds: 90061

Time = 1 days, 1 hours, 1 minutes, 1 seconds

Another Test

Enter time in seconds: 3661

Time = 0 days, 1 hours, 1 minutes, 1 seconds

PROGRAM 2: Check Whether a Year is a Leap Year

1. Aim

To write a C program to input a year and determine whether it is a leap year or not.

2. Logic

A year is a leap year if:

- It is divisible by 400, OR
- It is divisible by 4 but not divisible by 100.

Therefore:

$(\text{year} \% 400 == 0)$ OR

$(\text{year} \% 4 == 0 \text{ AND } \text{year} \% 100 != 0)$

3. Algorithm

1. Start.
2. Declare an integer variable year.
3. Input the year.
4. Check whether the year is divisible by 400.
5. If not, check whether it is divisible by 4 and not divisible by 100.
6. If either condition is true, display Leap Year.
7. Otherwise, display Not a Leap Year.
8. Stop.

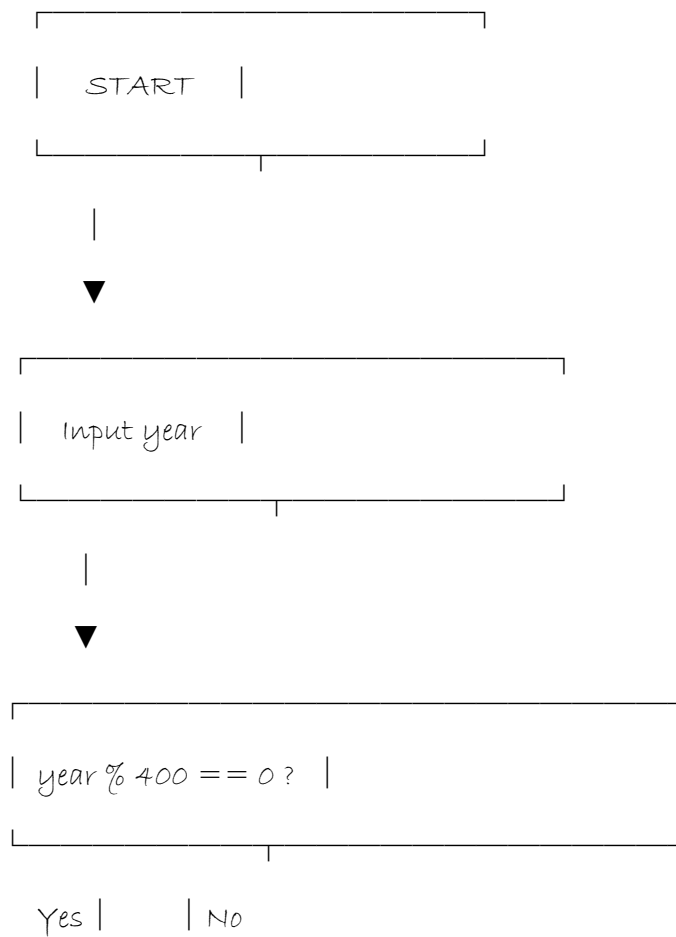
4. Pseudocode

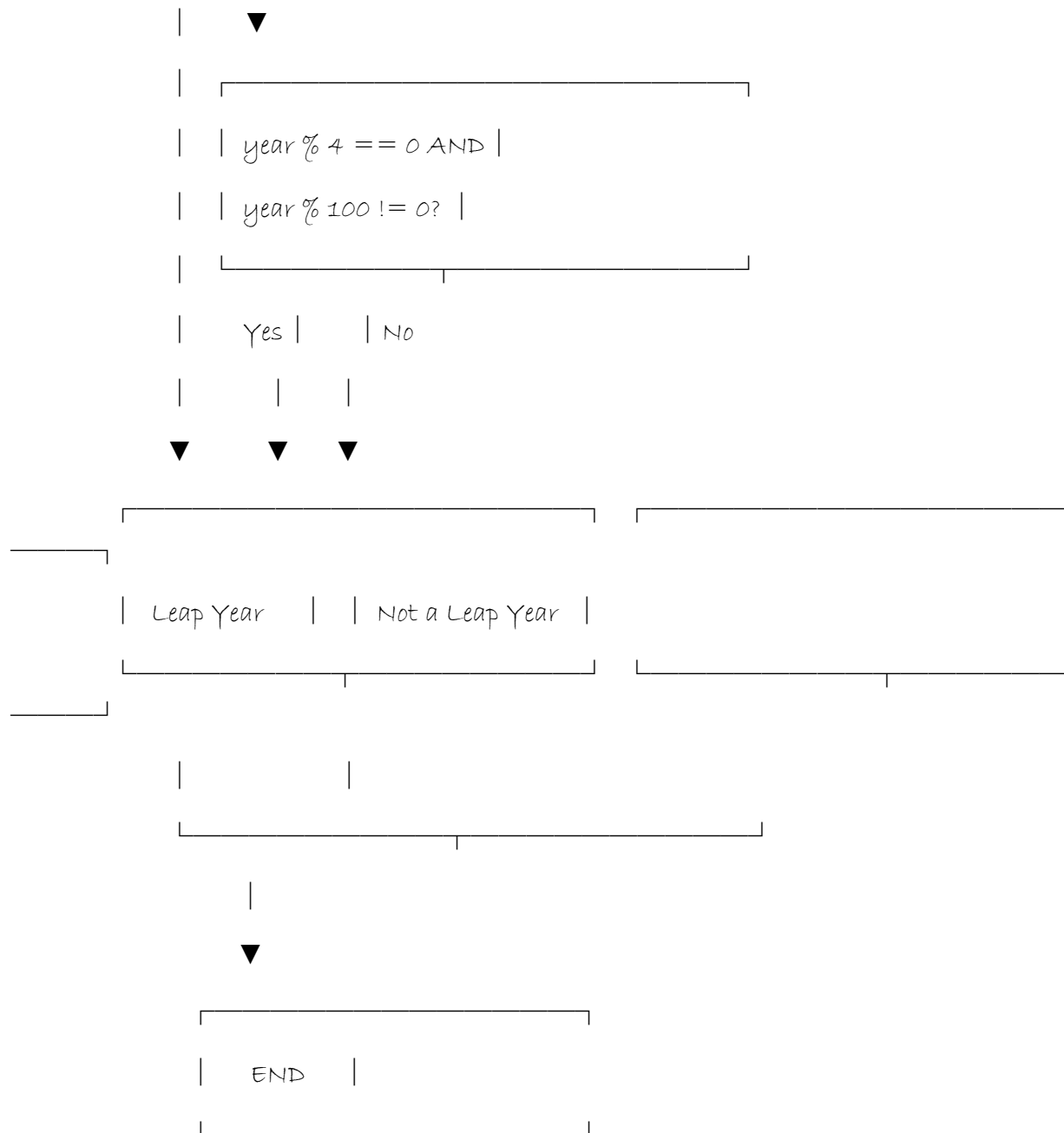
BEGIN

INPUT year

```
IF (year % 400 = 0) OR  
  (year % 4 = 0 AND year % 100 ≠ 0)  
THEN  
  DISPLAY "Leap Year"  
ELSE  
  DISPLAY "Not a Leap Year"  
END IF  
  
END
```

5. Flowchart





6. C Program

```
#include <stdio.h>
```

```
int main()
```

```

{
    int year;

    printf("Enter a year: ");
    scanf("%d", &year);

    if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0))
    {
        printf("%d is a Leap Year.\n", year);
    }
    else
    {
        printf("%d is Not a Leap Year.\n", year);
    }

    return 0;
}

```

7. Test Case Table

Test Case Input Year Expected Output

TC1	2024	Leap Year
TC2	2023	Not a Leap Year
TC3	2000	Leap Year
TC4	1900	Not a Leap Year

Test Case Input Year Expected Output

TC5 2028 Leap Year

TC6 2100 Not a Leap Year

Why these test cases?

- 2024 → divisible by 4 → Leap Year.
 - 2023 → not divisible by 4 → Not a Leap Year.
 - 2000 → divisible by 400 → Leap Year.
 - 1900 → divisible by 100 but not 400 → Not a Leap Year.
 - 2100 → divisible by 100 but not 400 → Not a Leap Year.
-

8. Compilation

Save the program as:

leap.c

Compile:

gcc leap.c -o leap

9. Execution

Run:

./leap

Test Case 1

Enter a year: 2024

2024 is a Leap Year.

Test Case 2

Enter a year: 1900

1900 is Not a Leap Year.

Test Case 3

Enter a year: 2000

2000 is a Leap Year.